INFOSYS SE ASSESSMENT 2026 • OFFICIAL PREPARATION VAULT

☕ Object-Oriented Programming (OOPs) Master Interview Vault

Section: 10_Interview_Preparation | **Drive Date:** 5th & 6th November 2026 | **Status:** 100% Solved & Verified

☕ Object-Oriented Programming (OOPs) Master Interview Vault

Core Focus: Java & C++ Concepts, Pillars, Architecture & Interview Traps

> **Infosys SE Interview Weightage:** Very High (Asked to almost 95% of engineering candidates).
> **Key Focus:** Clear real-world analogies, code syntax in Java/C++, and architectural differences.

---

1. The Four Pillars of OOPs

1.1 Encapsulation

- **Definition:** Wrapping code and data together into a single unit (class) and restricting direct access to some of the object's components (data hiding) using private access specifiers with public getters and setters.
- **Real-World Analogy:** A medical capsule where medicines are hidden inside the protective coating.
- **Java Implementation:**

```
public class Employee {
    private double salary; // Data hiding

    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        if (salary > 0) {
            this.salary = salary;
        }
    }
}
```

1.2 Abstraction

- **Definition:** Displaying only essential features to the user and hiding internal implementation details. Achieved via **Abstract Classes** (0–100% abstraction) and **Interfaces** (100% pure abstraction in Java 7, with default/static methods in Java 8+).

- **Real-World Analogy:** Driving a car where you use the steering wheel and brake pedals without needing to know the internal combustion or electronic transmission mechanics.

1.3 Inheritance

- **Definition:** Mechanism where one class acquires the properties and behaviors of a parent class, promoting code reusability.
- **Types:** Single, Multilevel, Hierarchical, Multiple (supported via Interfaces in Java to prevent the Diamond Problem), and Hybrid.

1.4 Polymorphism

- **Definition:** The ability of a message, function, or object to be displayed or executed in more than one form.
- **Compile-Time Polymorphism (Static Binding / Method Overloading):** Methods in the same class with identical names but different parameter lists (type, count, or sequence).
- **Runtime Polymorphism (Dynamic Binding / Method Overriding):** Subclass provides a specific implementation of a method already defined in its superclass using the `@Override` annotation.

---

2. High-Frequency Infosys Interview Questions

Q1: What is the difference between an Abstract Class and an Interface?

Feature	Abstract Class	Interface
Multiple Inheritance	No (Java classes can extend only one class)	Yes (A class can implement multiple interfaces)
Methods	Can have both abstract and concrete methods	Default to abstract (Java 8 allows `default` / `static`)
Variables	Can have instance variables with any access modifier	Only `public static final` constants
Speed	Faster than interface due to lookup mechanism	Slightly slower due to dynamic search

Q2: Why does Java not support Multiple Inheritance with Classes?

To avoid the **Diamond Problem** (Ambiguity). If class `D` extends both `B` and `C`, and both `B` and `C` override method `foo()` from class `A`, the compiler cannot determine which version of `foo()` class `D` should inherit.